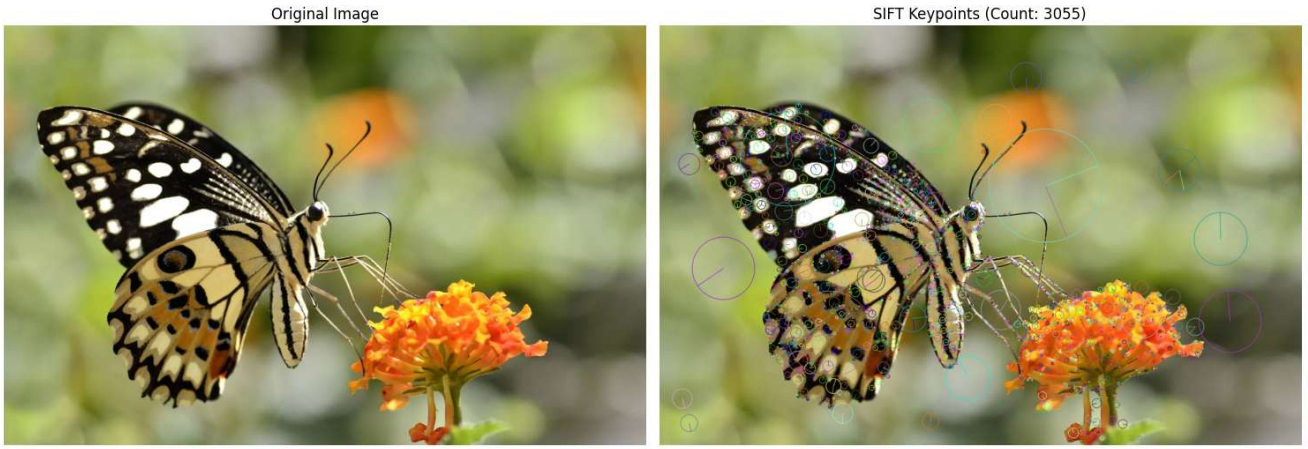
1. Build a keypoint detector and visualizer using SIFT to detect and display keypoints with their scale and orientation.

SHREYA CU24250018

```
import cv2
import matplotlib.pyplot as plt
import numpy as np
img_path = '/content/Lime-butterfly-Papilio-demoleus-feeding-on-flower.webp'
img = cv2.imread(img_path, cv2.IMREAD_COLOR)

if img is None:
    raise FileNotFoundError(f"Could not open or find the image at: {img_path}")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
sift = cv2.SIFT_create()
keypoints = sift.detect(gray, None)
img_with_keypoints = cv2.drawKeypoints(
    img,
    keypoints,
    None,
    flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS
)
fig, axes = plt.subplots(1, 2, figsize=(15, 8))
axes[0].imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
axes[0].set_title('Original Image')
axes[0].axis('off')
axes[1].imshow(cv2.cvtColor(img_with_keypoints, cv2.COLOR_BGR2RGB))
axes[1].set_title(f'SIFT Keypoints (Count: {len(keypoints)})')
axes[1].axis('off')

plt.tight_layout()
plt.show()
```

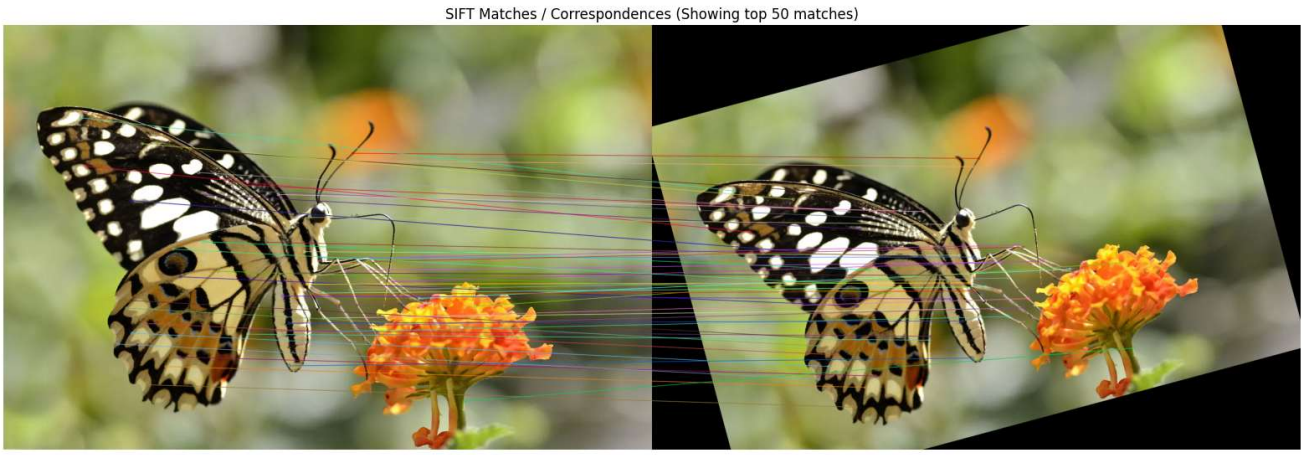


2. Use SIFT descriptors to find and draw correspondences between two photos of the same scene.

SHREYA CU24250018

```
import cv2
import matplotlib.pyplot as plt
import numpy as np
img1 = cv2.imread('/content/Lime-butterfly-Papilio-demoleus-feeding-on-flower.webp', cv2.IMREAD_COLOR)
if img1 is None:
    raise FileNotFoundError("Could not load the image. Please make sure the path is correct.")
h, w = img1.shape[:2]
M = cv2.getRotationMatrix2D((w / 2, h / 2), 15, 0.9)
img2 = cv2.warpAffine(img1, M, (w, h))
gray1 = cv2.cvtColor(img1, cv2.COLOR_BGR2GRAY)
gray2 = cv2.cvtColor(img2, cv2.COLOR_BGR2GRAY)
sift = cv2.SIFT_create()
```

```
kp1, des1 = sift.detectAndCompute(gray1, None)
kp2, des2 = sift.detectAndCompute(gray2, None)
FLANN_INDEX_KDTREE = 1
index_params = dict(algorithm=FLANN_INDEX_KDTREE, trees=5)
search_params = dict(checks=50)
flann = cv2.FlannBasedMatcher(index_params, search_params)
matches = flann.knnMatch(des1, des2, k=2)
good_matches = []
for m, n in matches:
    if m.distance < 0.7 * n.distance:
        good_matches.append(m)
good_matches = sorted(good_matches, key=lambda x: x.distance)[:50]
img_matches = cv2.drawMatches(
    img1, kp1,
    img2, kp2,
    good_matches,
    None,
    flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS
)
plt.figure(figsize=(20, 10))
plt.imshow(cv2.cvtColor(img_matches, cv2.COLOR_BGR2RGB))
plt.title(f'SIFT Matches / Correspondences (Showing top {len(good_matches)} matches)')
plt.axis('off')
plt.show()
```



3. Create a panorama by matching SIFT features between two images and using homography.

SHREYA CU24250018

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Helper function modified to load local images directly
def load_image(path):
    image = cv2.imread(path, cv2.IMREAD_COLOR)
    if image is None:
        raise FileNotFoundError(f"Could not find or load image at {path}")
    return image

# Local file paths
path1 = '/content/Lime-butterfly-Papilio-demoleus-feeding-on-flower.webp'

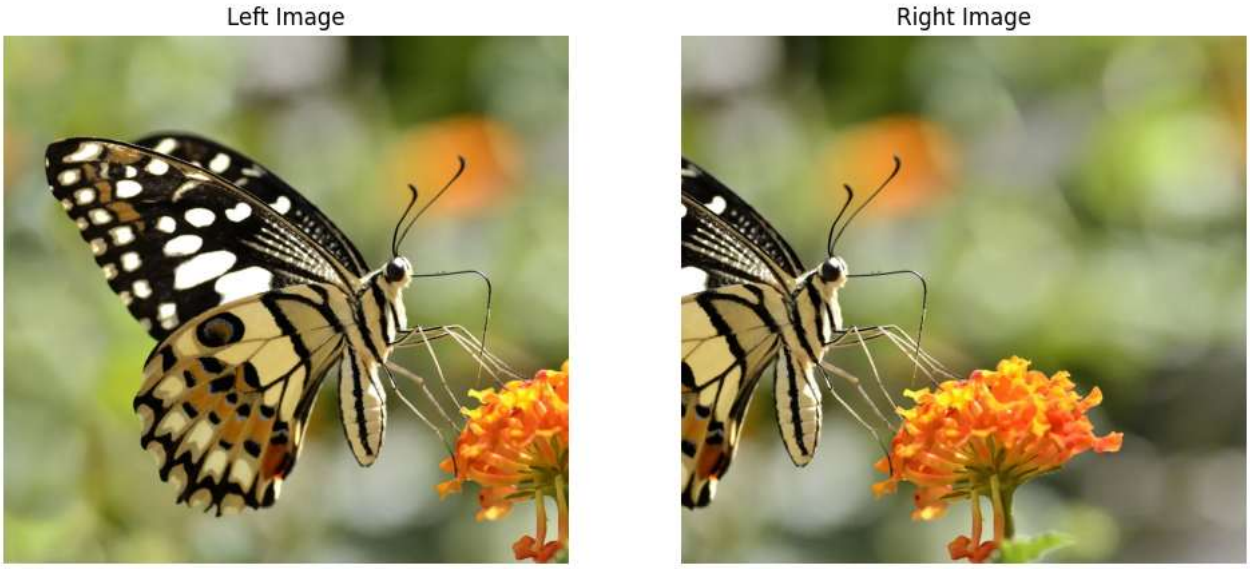
# Load original image
img_base = load_image(path1)
h, w = img_base.shape[:2]

# Create left and right overlapping images for the panorama
# Left image: Crop left 70%
img_left = img_base[:, :int(w * 0.7)]
```

```
# Right image: Apply a slight homography/perspective transform and crop right 70%
M = np.float32([[1, 0, -int(w * 0.3)], [0, 1, 0]])
img_right = cv2.warpAffine(img_base, M, (int(w * 0.7), h))

# Convert to RGB for visualization
img_left_rgb = cv2.cvtColor(img_left, cv2.COLOR_BGR2RGB)
img_right_rgb = cv2.cvtColor(img_right, cv2.COLOR_BGR2RGB)

fig, axes = plt.subplots(1, 2, figsize=(12, 6))
axes[0].imshow(img_left_rgb)
axes[0].set_title('Left Image')
axes[0].axis('off')
axes[1].imshow(img_right_rgb)
axes[1].set_title('Right Image')
axes[1].axis('off')
plt.show()
```



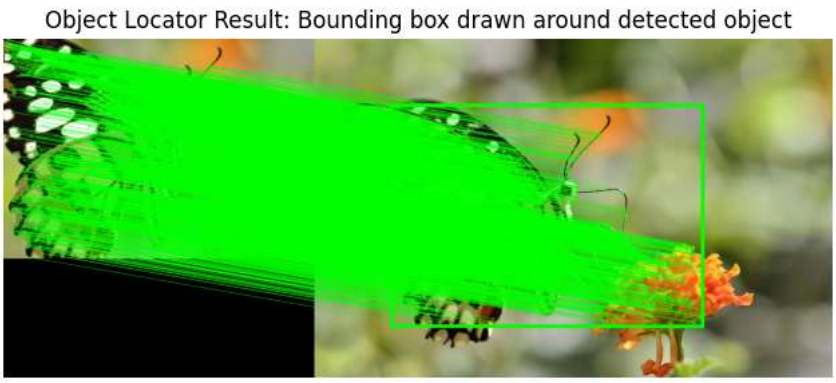

```
sift = cv2.SIFT_create()
kp_left, des_left = sift.detectAndCompute(img_left, None)
kp_right, des_right = sift.detectAndCompute(img_right, None)
FLANN_INDEX_KDTREE = 1
index_params = dict(algorithm=FLANN_INDEX_KDTREE, trees=5)
search_params = dict(checks=50)
flann = cv2.FlannBasedMatcher(index_params, search_params)
matches = flann.knnMatch(des_right, des_left, k=2)
good_matches = []
for m, n in matches:
    if m.distance < 0.7 * n.distance:
        good_matches.append(m)
if len(good_matches) > 4:
    src_pts = np.float32([kp_right[m.queryIdx].pt for m in good_matches]).reshape(-1, 1, 2)
    dst_pts = np.float32([kp_left[m.trainIdx].pt for m in good_matches]).reshape(-1, 1, 2)
    H, mask = cv2.findHomography(src_pts, dst_pts, cv2.RANSAC, 5.0)
    panorama_width = img_left.shape[1] + img_right.shape[1]
    panorama_height = img_left.shape[0]
    warped_right = cv2.warpPerspective(img_right, H, (panorama_width, panorama_height))
    panorama = warped_right.copy()
    panorama[0:img_left.shape[0], 0:img_left.shape[1]] = img_left
    gray_panorama = cv2.cvtColor(panorama, cv2.COLOR_BGR2GRAY)
    _, thresh = cv2.threshold(gray_panorama, 1, 255, cv2.THRESH_BINARY)
    contours, _ = cv2.findContours(thresh, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
    x, y, w_box, h_box = cv2.boundingRect(contours[0])
    panorama_cropped = panorama[y:y+h_box, x:x+w_box]
    plt.figure(figsize=(8,8))
    plt.imshow(cv2.cvtColor(panorama_cropped, cv2.COLOR_BGR2RGB))
    plt.title('Final Stitched Panorama')
    plt.axis('off')
    plt.show()
else:
    print(f"Not enough keypoint matches found - {len(good_matches)}/4 required.")
```



4. Build a logo/object locator in a cluttered scene using SIFT feature matching and homography.

SHREYA CU24250018

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
scene_img = cv2.imread('/content/Lime-butterfly-Papilio-demoleus-feeding-on-flower.webp', cv2.IMREAD_COLOR)
if scene_img is None:
    raise FileNotFoundError("Cluttered scene image could not be loaded.")
h, w = scene_img.shape[:2]
template_img = scene_img[int(h*0.2):int(h*0.85), int(w*0.15):int(w*0.75)]
gray_template = cv2.cvtColor(template_img, cv2.COLOR_BGR2GRAY)
gray_scene = cv2.cvtColor(scene_img, cv2.COLOR_BGR2GRAY)
sift = cv2.SIFT_create()
kp_template, des_template = sift.detectAndCompute(gray_template, None)
kp_scene, des_scene = sift.detectAndCompute(gray_scene, None)
FLANN_INDEX_KDTREE = 1
index_params = dict(algorithm=FLANN_INDEX_KDTREE, trees=5)
search_params = dict(checks=50)
flann = cv2.FlannBasedMatcher(index_params, search_params)
matches = flann.knnMatch(des_template, des_scene, k=2)
good_matches = []
for m, n in matches:
    if m.distance < 0.7 * n.distance:
        good_matches.append(m)
MIN_MATCH_COUNT = 10
if len(good_matches) > MIN_MATCH_COUNT:
    src_pts = np.float32([kp_template[m.queryIdx].pt for m in good_matches]).reshape(-1, 1, 2)
    dst_pts = np.float32([kp_scene[m.trainIdx].pt for m in good_matches]).reshape(-1, 1, 2)
    H, mask = cv2.findHomography(src_pts, dst_pts, cv2.RANSAC, 5.0)
    th, tw = template_img.shape[:2]
    pts = np.float32([[0, 0], [0, th-1], [tw-1, th-1], [tw-1, 0]]).reshape(-1, 1, 2)
    dst_corners = cv2.perspectiveTransform(pts, H)
    detected_scene = scene_img.copy()
    detected_scene = cv2.polylines(detected_scene, [np.int32(dst_corners)], True, (0, 255, 0), 10, cv2.LINE_AA)
    draw_params = dict(matchColor=(0, 255, 0), singlePointColor=None, matchesMask=mask.ravel().tolist(), flags=2)
    img_matching_result = cv2.drawMatches(template_img, kp_template, detected_scene, kp_scene, good_matches, None, **draw_param
    plt.figure(figsize=(8,8))
    plt.imshow(cv2.cvtColor(img_matching_result, cv2.COLOR_BGR2RGB))
    plt.title('Object Locator Result: Bounding box drawn around detected object')
    plt.axis('off')
    plt.show()
else:
    print(f"Not enough matches were found to localize the object: {len(good_matches)}/{MIN_MATCH_COUNT}")
```



5. Implement a HOG-based pedestrian/person detector demo and extract HOG features.

SHREYA CU24250018

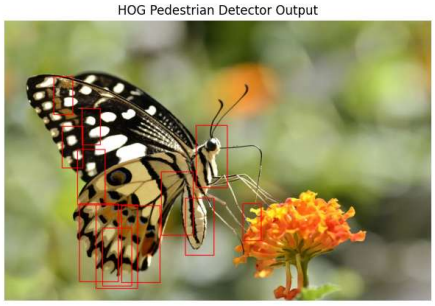
```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from skimage.feature import hog
from skimage import exposure
img_path = '/content/Lime-butterfly-Papilio-demoleus-feeding-on-flower.webp'
image = cv2.imread(img_path, cv2.IMREAD_COLOR)
gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
hog_detector = cv2.HOGDescriptor()
hog_detector.setSVMDetector(cv2.HOGDescriptor_getDefaultPeopleDetector())
rects, weights = hog_detector.detectMultiScale(gray_image, winStride=(4, 4), padding=(8, 8), scale=1.05)
detected_img = image.copy()
for (x, y, w, h) in rects:
    cv2.rectangle(detected_img, (x, y), (x + w, y + h), (0, 0, 255), 2)
h, w = gray_image.shape[:2]
crop_patch = gray_image[int(h*0.3):int(h*0.3)+256, int(w*0.3):int(w*0.3)+256]

features, hog_viz = hog(
    crop_patch,
    orientations=9,
    pixels_per_cell=(8, 8),
    cells_per_block=(2, 2),
    visualize=True,
    channel_axis=None
)
hog_viz_rescaled = exposure.rescale_intensity(hog_viz, in_range=(0, 10))
fig, axes = plt.subplots(1, 3, figsize=(18, 6))
axes[0].imshow(cv2.cvtColor(detected_img, cv2.COLOR_BGR2RGB))
axes[0].set_title('HOG Pedestrian Detector Output')
axes[0].axis('off')

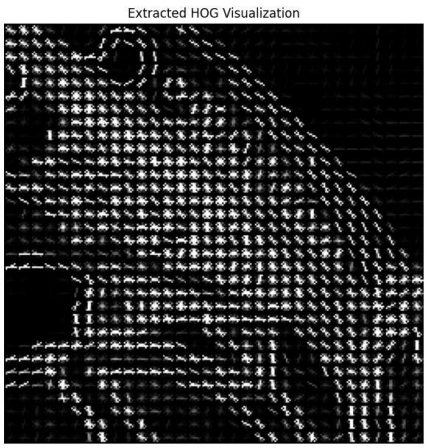
axes[1].imshow(crop_patch, cmap='gray')
axes[1].set_title('Cropped Patch for HOG Extraction')
axes[1].axis('off')

axes[2].imshow(hog_viz_rescaled, cmap='gray')
axes[2].set_title('Extracted HOG Visualization')
axes[2].axis('off')

plt.tight_layout()
plt.show()
```







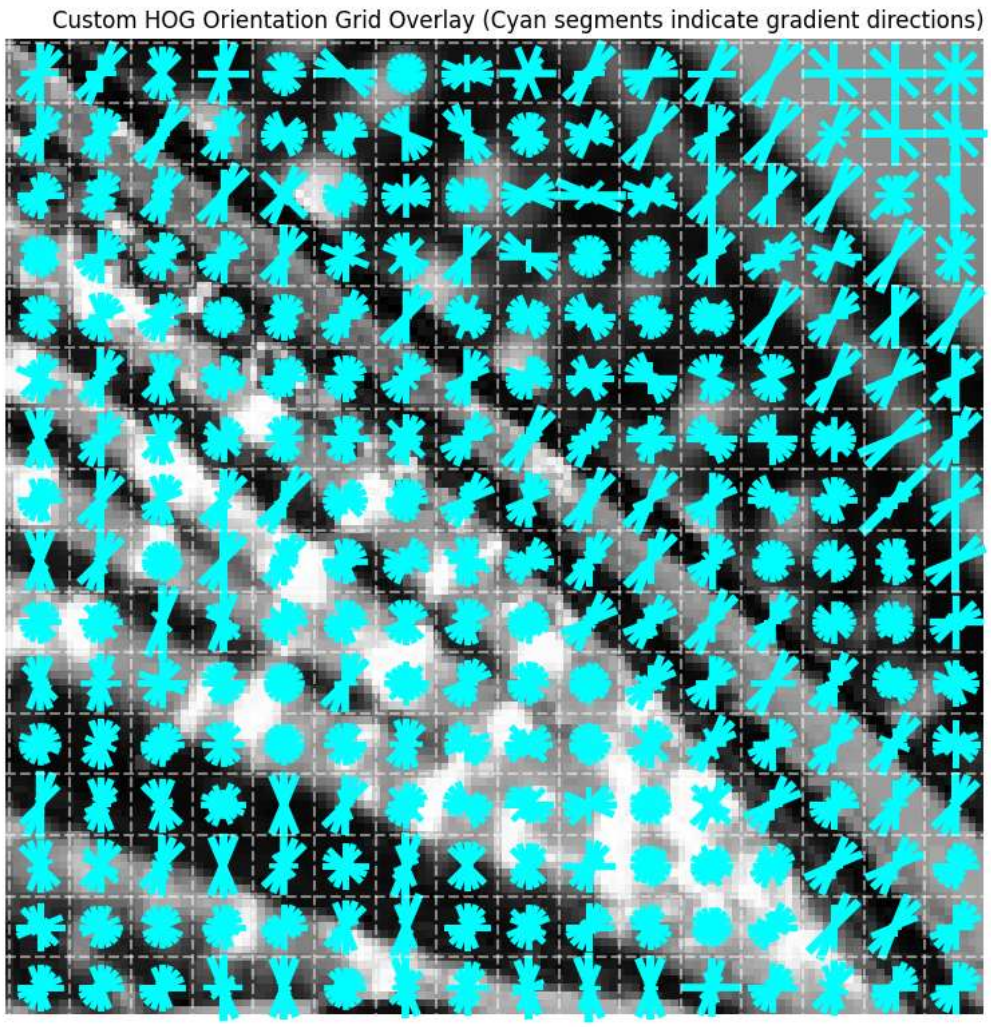
6. Create a HOG visualizer that displays the gradient-orientation histogram grid over the image.

SHREYA CU24250018

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from skimage.feature import hog
img = cv2.imread('/content/Lime-butterfly-Papilio-demoleus-feeding-on-flower.webp', cv2.IMREAD_COLOR)
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
h, w = gray.shape[:2]
crop_patch = gray[int(h*0.35):int(h*0.35)+128, int(w*0.35):int(w*0.35)+128]
orientations = 8
pixels_per_cell = (8,8)
cells_per_block = (1, 1)
fd, hog_image = hog(
    crop_patch,
    orientations=orientations,
    pixels_per_cell=pixels_per_cell,
    cells_per_block=cells_per_block,
    visualize=True,
    channel_axis=None
)
n_cells_y = crop_patch.shape[0] // pixels_per_cell[0]
n_cells_x = crop_patch.shape[1] // pixels_per_cell[1]
fd_reshaped = fd.reshape(n_cells_y, n_cells_x, orientations)

fig, ax = plt.subplots(figsize=(10, 10))
ax.imshow(crop_patch, cmap='gray')
for r in range(n_cells_y):
    for c in range(n_cells_x):
        cx = c * pixels_per_cell[1] + pixels_per_cell[1] / 2
        cy = r * pixels_per_cell[0] + pixels_per_cell[0] / 2
        for o in range(orientations):
            angle = (o * np.pi) / orientations
            magnitude = fd_reshaped[r, c, o]
            dx = magnitude * np.cos(angle) * pixels_per_cell[1] * 0.8
            dy = magnitude * np.sin(angle) * pixels_per_cell[0] * 0.8
            ax.plot([cx - dx, cx + dx], [cy - dy, cy + dy], color='cyan', lw=1.5 + magnitude * 5)
for x in range(0, crop_patch.shape[1], pixels_per_cell[1]):
    ax.axvline(x, color='white', linestyle='--', alpha=0.5)
for y in range(0, crop_patch.shape[0], pixels_per_cell[0]):
    ax.axhline(y, color='white', linestyle='--', alpha=0.5)

ax.set_title('Custom HOG Orientation Grid Overlay (Cyan segments indicate gradient directions)')
ax.axis('off')
plt.show()
```



7. Build a duplicate/near-duplicate image detector using SIFT similarity matching.

SHREYA CU24250018

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

def detect_duplicates(img1_path, img2_path, match_ratio_threshold=0.7, similarity_threshold=0.15):
    img1 = cv2.imread(img1_path, cv2.IMREAD_GRAYSCALE)
    img2 = cv2.imread(img2_path, cv2.IMREAD_GRAYSCALE)
    if img1 is None or img2 is None:
        raise FileNotFoundError("One of the images could not be loaded.")
    sift = cv2.SIFT_create()
    kp1, des1 = sift.detectAndCompute(img1, None)
    kp2, des2 = sift.detectAndCompute(img2, None)

    if des1 is None or des2 is None:
        return False, 0.0, "No descriptors found."
    FLANN_INDEX_KDTREE = 1
    index_params = dict(algorithm=FLANN_INDEX_KDTREE, trees=5)
    search_params = dict(checks=50)
    flann = cv2.FlannBasedMatcher(index_params, search_params)
    matches = flann.knnMatch(des1, des2, k=2)
    good_matches = []
    for m, n in matches:
        if m.distance < match_ratio_threshold * n.distance:
            good_matches.append(m)
    min_kps = min(len(kp1), len(kp2))
    if min_kps == 0:
        return False, 0.0, "Empty keypoints list."
```

```
    similarity_score = len(good_matches) / min_kps
    is_duplicate = False
    msg = "Images are not duplicates."

    if len(good_matches) >= 8:
        src_pts = np.float32([kp1[m.queryIdx].pt for m in good_matches]).reshape(-1, 1, 2)
        dst_pts = np.float32([kp2[m.trainIdx].pt for m in good_matches]).reshape(-1, 1, 2)

        H, mask = cv2.findHomography(src_pts, dst_pts, cv2.RANSAC, 5.0)
        if mask is not None:
            inlier_ratio = np.sum(mask) / len(good_matches)
            if similarity_score >= similarity_threshold and inlier_ratio > 0.5:
                is_duplicate = True
                msg = f"Near-Duplicate/Duplicate Detected! (Similarity Score: {similarity_score:.2f}, Inlier Ratio: {inlier_rat
    else:
        msg = f"Not enough good matches to verify geometry. (Good matches: {len(good_matches)})"

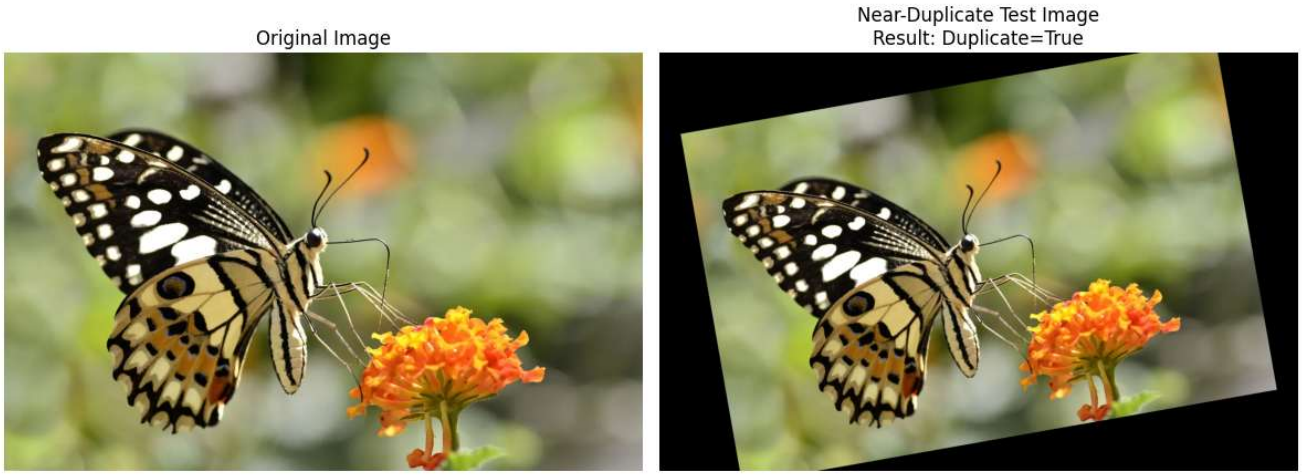
    return is_duplicate, similarity_score, msg
img_base = cv2.imread('/content/Lime-butterfly-Papilio-demoleus-feeding-on-flower.webp')
h, w = img_base.shape[:2]
M = cv2.getRotationMatrix2D((w/2, h/2), 10, 0.85)
near_duplicate_img = cv2.warpAffine(img_base, M, (w, h))
cv2.imwrite('/content/near_duplicate_test.webp', near_duplicate_img)
is_dup, score, output_message = detect_duplicates(
    '/content/Lime-butterfly-Papilio-demoleus-feeding-on-flower.webp',
    '/content/near_duplicate_test.webp'
)
print(f"Result: {output_message}")

fig, axes = plt.subplots(1, 2, figsize=(12, 6))
axes[0].imshow(cv2.cvtColor(img_base, cv2.COLOR_BGR2RGB))
axes[0].set_title("Original Image")
axes[0].axis("off")

axes[1].imshow(cv2.cvtColor(near_duplicate_img, cv2.COLOR_BGR2RGB))
axes[1].set_title(f"Near-Duplicate Test Image\nResult: Duplicate={is_dup}")
axes[1].axis("off")

plt.tight_layout()
plt.show()
```

```
Result: Near-Duplicate/Duplicate Detected! (Similarity Score: 0.71, Inlier Ratio: 0.98)
```



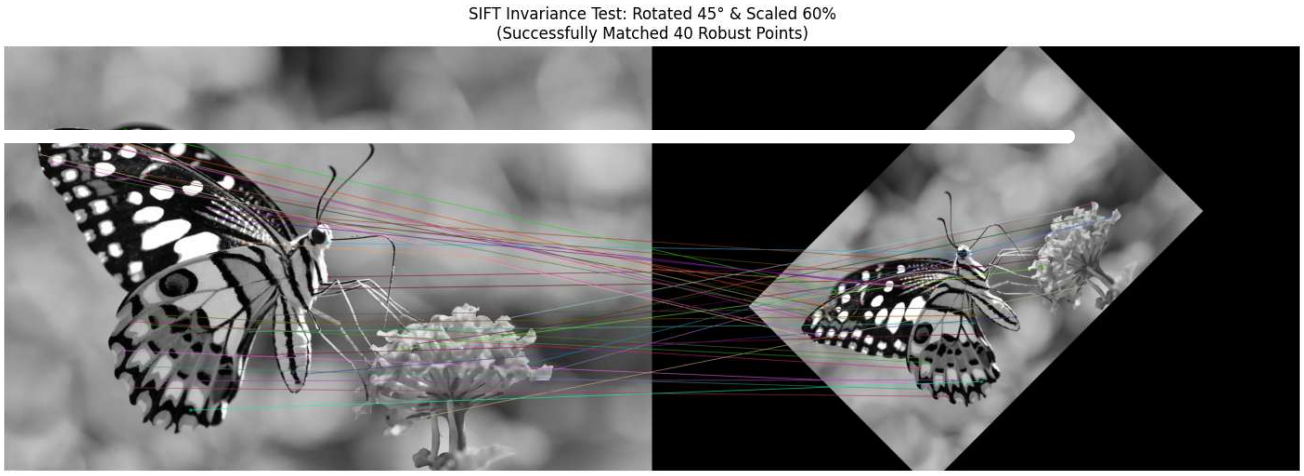
8. Test SIFT's rotation and scale invariance by applying rotation and scaling transformations to an image.

SHREYA CU24250018

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
img_src = cv2.imread('/content/Lime-butterfly-Papilio-demoleus-feeding-on-flower.webp', cv2.IMREAD_GRAYSCALE)
h, w = img_src.shape[:2]
scale_factor = 0.6
```

```
rotation_angle = 45
M_transform = cv2.getRotationMatrix2D((w / 2, h / 2), rotation_angle, scale_factor)
img_transformed = cv2.warpAffine(img_src, M_transform, (w, h))
sift = cv2.SIFT_create()
kp_orig, des_orig = sift.detectAndCompute(img_src, None)
kp_trans, des_trans = sift.detectAndCompute(img_transformed, None)
FLANN_INDEX_KDTREE = 1
index_params = dict(algorithm=FLANN_INDEX_KDTREE, trees=5)
search_params = dict(checks=50)
flann = cv2.FlannBasedMatcher(index_params, search_params)
matches = flann.knnMatch(des_orig, des_trans, k=2)
good_matches = []
for m, n in matches:
    if m.distance < 0.7 * n.distance:
        good_matches.append(m)
good_matches = sorted(good_matches, key=lambda x: x.distance)[:40]
img_matches_visualization = cv2.drawMatches(
    img_src, kp_orig,
    img_transformed, kp_trans,
    good_matches,
    None,
    flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS
)

plt.figure(figsize=(18, 9))
plt.imshow(img_matches_visualization, cmap='gray')
plt.title(f"SIFT Invariance Test: Rotated {rotation_angle}° & Scaled {int(scale_factor*100)}%\n(Successfully Matched {len(good_
plt.axis('off')
plt.show()
```



9. Build an image retrieval system that ranks images in a small database according to SIFT similarity with a query image.

SHREYA CU24250018

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
import os
db_dir = '/content/image_db'
os.makedirs(db_dir, exist_ok=True)
img_src = cv2.imread('/content/Lime-butterfly-Papilio-demoleus-feeding-on-flower.webp')
h, w = img_src.shape[:2]
cv2.imwrite(os.path.join(db_dir, 'db_img1_cropped.webp'), img_src[int(h*0.1):int(h*0.9), int(w*0.1):int(w*0.9)])
rotated_90 = cv2.rotate(img_src, cv2.ROTATE_90_CLOCKWISE)
cv2.imwrite(os.path.join(db_dir, 'db_img2_rotated.webp'), rotated_90)
noise_img = np.random.randint(0, 255, (h, w, 3), dtype=np.uint8)
cv2.imwrite(os.path.join(db_dir, 'db_img3_unrelated.webp'), noise_img
rescaled = cv2.resize(img_src, (0, 0), fx=0.4, fy=0.4)
cv2.imwrite(os.path.join(db_dir, 'db_img4_scaled.webp'), rescaled)
```

```
def retrieve_images(query_img_path, database_dir):
    query_img = cv2.imread(query_img_path, cv2.IMREAD_GRAYSCALE)
    sift = cv2.SIFT_create()
    kp_q, des_q = sift.detectAndCompute(query_img, None)

    if des_q is None:
        print("No descriptors found in query image.")
        return []
    FLANN_INDEX_KDTREE = 1
    index_params = dict(algorithm=FLANN_INDEX_KDTREE, trees=5)
    search_params = dict(checks=50)
    flann = cv2.FlannBasedMatcher(index_params, search_params)

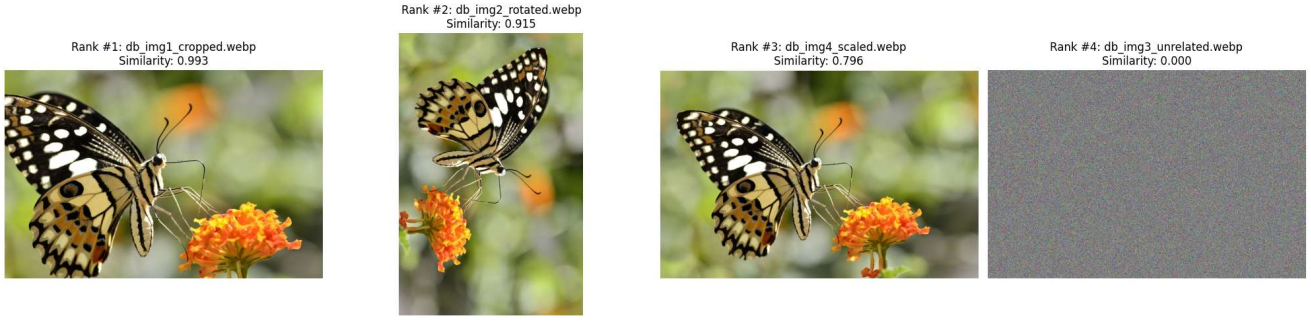
    results = []
    for filename in os.listdir(database_dir):
        db_path = os.path.join(database_dir, filename)
        db_img = cv2.imread(db_path, cv2.IMREAD_GRAYSCALE)

        if db_img is None:
            continue

        kp_db, des_db = sift.detectAndCompute(db_img, None)
        if des_db is None:
            results.append((filename, 0.0, db_path))
            continue
        matches = flann.knnMatch(des_q, des_db, k=2)
        good_matches = []
        for match in matches:
            if len(match) == 2:
                m, n = match
                if m.distance < 0.7 * n.distance:
                    good_matches.append(m)
        min_kps = min(len(kp_q), len(kp_db))
        score = len(good_matches) / min_kps if min_kps > 0 else 0.0

        results.append((filename, score, db_path))
    results = sorted(results, key=lambda x: x[1], reverse=True)
    return results
query_path = '/content/Lime-butterfly-Papilio-demoleus-feeding-on-flower.webp'
ranked_results = retrieve_images(query_path, db_dir)
fig, axes = plt.subplots(1, len(ranked_results), figsize=(20, 5))
for i, (name, score, path) in enumerate(ranked_results):
    retrieved_bgr = cv2.imread(path)
    retrieved_rgb = cv2.cvtColor(retrieved_bgr, cv2.COLOR_BGR2RGB)
    axes[i].imshow(retrieved_rgb)
    axes[i].set_title(f"Rank #{i+1}: {name}\nSimilarity: {score:.3f}")
    axes[i].axis('off')

plt.tight_layout()
plt.show()
```



10. Compare SIFT and HOG feature descriptors based on feature/keypoint information and extraction time.

SHREYA CU24250018

```
import time
import cv2
import numpy as np
```

```
from skimage.feature import hog
img_color = cv2.imread('/content/Lime-butterfly-Papilio-demoleus-feeding-on-flower.webp')
gray_img = cv2.cvtColor(img_color, cv2.COLOR_BGR2GRAY)
t0 = time.time()
sift = cv2.SIFT_create()
kps, des_sift = sift.detectAndCompute(gray_img, None)
sift_time = (time.time() - t0) * 1000
t0 = time.time()
hog_features = hog(
    gray_img,
    orientations=9,
    pixels_per_cell=(16, 16),
    cells_per_block=(2, 2),
    visualize=False
)
hog_time = (time.time() - t0) * 1000
print(f"--- SIFT Feature Extraction ---")
print(f"Keypoints detected: {len(kps)}")
print(f"Descriptor shape: {des_sift.shape if des_sift is not None else 'None'}")
print(f"Extraction Time: {sift_time:.2f} ms\n")

print(f"--- HOG Feature Extraction ---")
print(f"Total HOG Features (flat vector size): {len(hog_features)}")
print(f"Extraction Time: {hog_time:.2f} ms\n")
```

```
--- SIFT Feature Extraction ---
Keypoints detected: 3055
Descriptor shape: (3055, 128)
Extraction Time: 295.49 ms

--- HOG Feature Extraction ---
Total HOG Features (flat vector size): 228096
Extraction Time: 233.47 ms
```

SIFT vs. HOG Feature Comparison Summary

Feature Property	SIFT (Scale-Invariant Feature Transform)	HOG (Histogram of Ori
Core Concept	Detects local, salient keypoints (blobs) using Difference of Gaussians (DoG) and computes a localized 128D gradient histogram descriptor.	Computes dense local g
Invariances	Robustly invariant to **scale, rotation**, and minor affine/perspective distortions.	Partially invariant to min